Lab Report No 2
Sequence in Python
Digital Signal Processing

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted To:

Engg. Hareem Khan

Dated:

[1],2024

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1(i) :

Solution:

Brief description (3-5 lines)

Run all above lab examples and experiment by changing time interval range, step values.

Unit Step Sequence:

$u[n] = \{ 1, n \geq 0$

$0, n < 0$

The unit step function, denoted by u[n]. It is defined such that it is zero for n<0 and for all the positive values of n including n = 0, its value is one.

Unit Impulse Signal:

$\delta[n] = \{ 1, n = 0$

$0, n \neq 0$

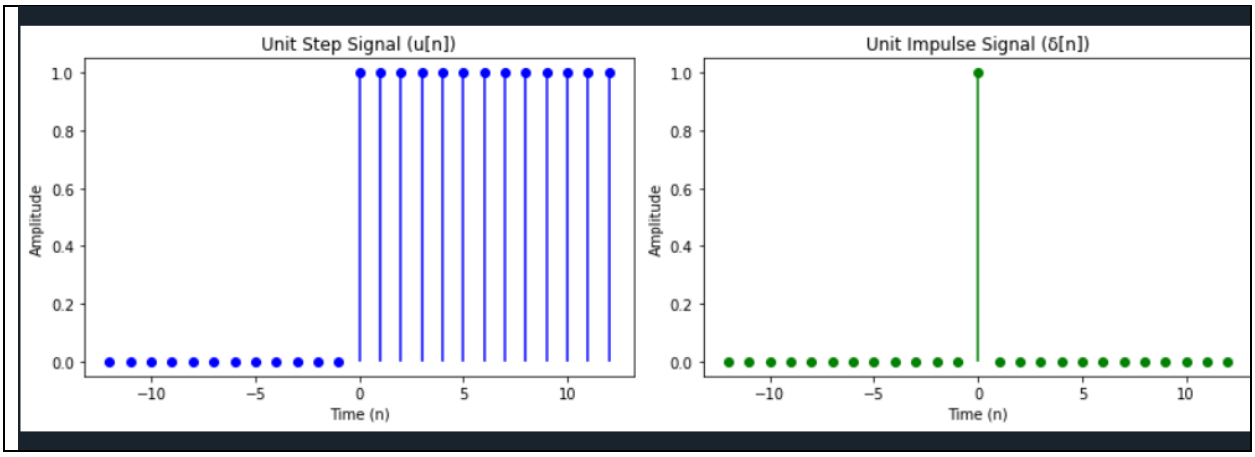
The unit impulse function, denoted by δ[n], is also known as the Dirac delta function. It is defined

such that it is zero everywhere except at n = 0, where its value is one.

The code

```
#unit step sequence and unit impluse signal
import numpy as np
import matplotlib.pyplot as plt
n=np.arrange(-11,12)
u=np.where(n>=1,0,1)
delta=np.where(n==1,0,1)
plt.figure(figsize=(15,5))
plt.subplot(1,2,1)
plt.stem(n,u,'b',marketfmt='bo',basefmt="",use_line_collection=True)
plt.title('Unit Step Signal(u[n]')
plt.xlabel('Time(n)')
plt.ylabel('Amplitude')
plt.subplot(1,2,1)
plt.stem(n,delta,'g',markerfmt='go',basefmt="",ues_line_collection=True)
plt.title('Unit Impluse Signal(δ[n])')
plt.xlabel('Time(n)')
plt.ylabel("Ampltiude")
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Task 1(B) :

Solution:

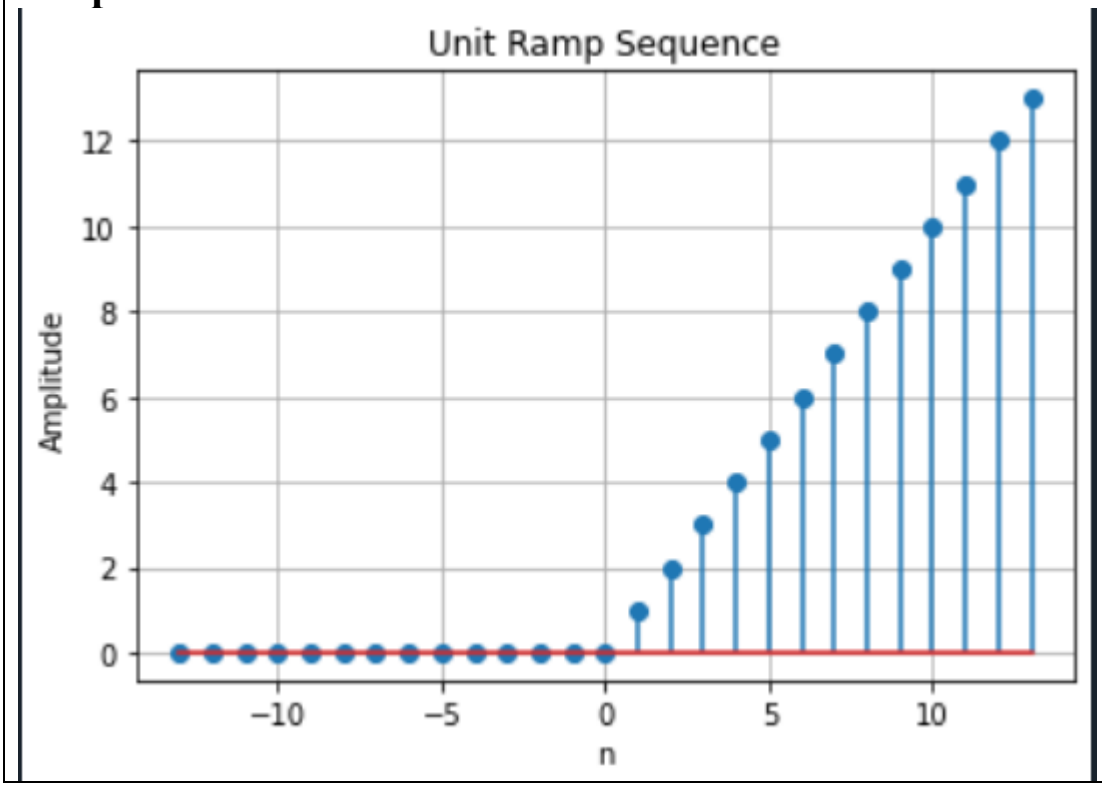
Brief description (3-5 lines)
Run all above lab examples and experiment by changing time interval range, step values. **Unit Ramp Signal:** The unit ramp function is also used to define other functions, such as the step function and the impulse function. The step function is defined as the derivative of the ramp function, while the impulse function is defined as the second derivative of the ramp function. The unit ramp function starts at 0 for negative values of n and increases linearly with time for n >= 0. r[n] = { n, n > 0 0, n ≤ 0 It is commonly used to represent a signal that starts from zero and increases linearly with time,such as the position of an object that is moving with a constant velocity.
The code

```python
# Unit Ramp Signal
import numpy as np
import matplotlib.pyplot as plt
n=np.arrange(-14,15)
print(n)
ramp=np.maximum(0,n)
plt.stem(n,ramp)
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.title('Unit Ramp Sequence')
plt.grid(True)
plt.show()
```

The results (Screenshot)

```
In [7]: runfile('D:/BSCOMPUTER ENGINEERING/3rd YEAR OF COMPUTER ENGINEERING/6th SEMESTER/DIGITAL SIGNAL PROCESSING/PROGRAMS OF DIGITAL SIGNAL PROCESSING/UNITRAMPSEQUENCE.PY', wdir='D:/BSCOMPUTER ENGINEERING/3rd YEAR OF COMPUTER ENGINEERING/6th SEMESTER/DIGITAL SIGNAL PROCESSING/PROGRAMS OF DIGITAL SIGNAL PROCESSING')
[-13 -12 -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4
   5   6   7   8   9  10  11  12  13]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  2  3  4  5  6  7  8  9 10
 11 12 13]
```

Graph:



Lab Task No 1(C) :

Solution:

Brief description (3-5 lines)

Run all above lab examples and experiment by changing time interval range, step values.

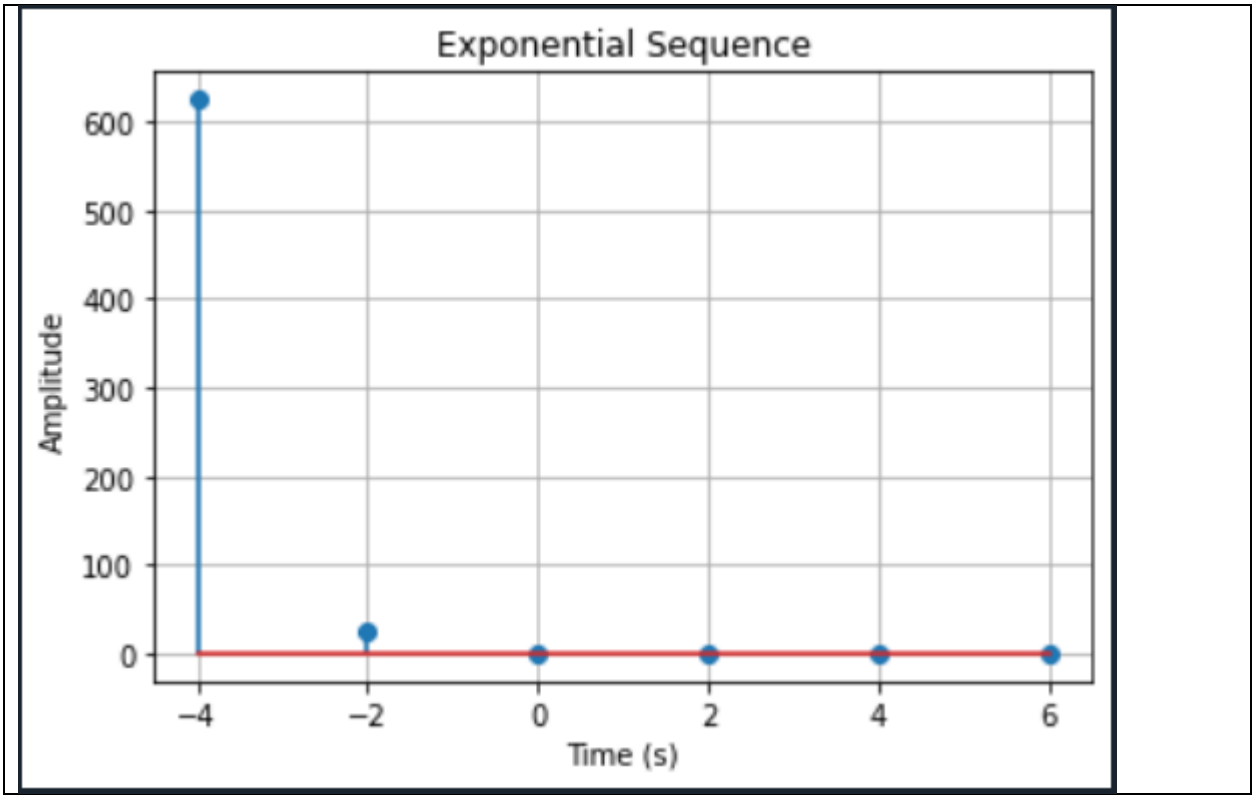
Exponential Sequences:

In digital signal processing, an exponential sequence is a discrete-time signal that exhibits exponential behavior. It is characterized by a real or complex number raised to the power of the index. The exponential sequence can be either growing or decaying, depending on the growth or decay factor r.For real-valued exponential sequences, the general form is x(n) = A * r^n, where A is the initial value, and r is the growth or decay factor. If r > 1, the sequence grows exponentially. If r < 1, the sequence decays exponentially. If r = 1, the sequence is constant.

The code

```
1   # Exponential Sequences
2   import numpy as np
3   import matplotlib.pyplot as plt
4   a = 0.2
5   n = np.arange(-4, 7, 2.0)
6   x = a**n
7   plt.stem(n, x)
8   plt.xlabel('Time (s)')
9   plt.ylabel('Amplitude')
10  plt.title('Exponential Sequence')
11  plt.grid(True)
12  plt.show()
```

The results (Screenshot)



Lab Task No 1(d):

Solution:

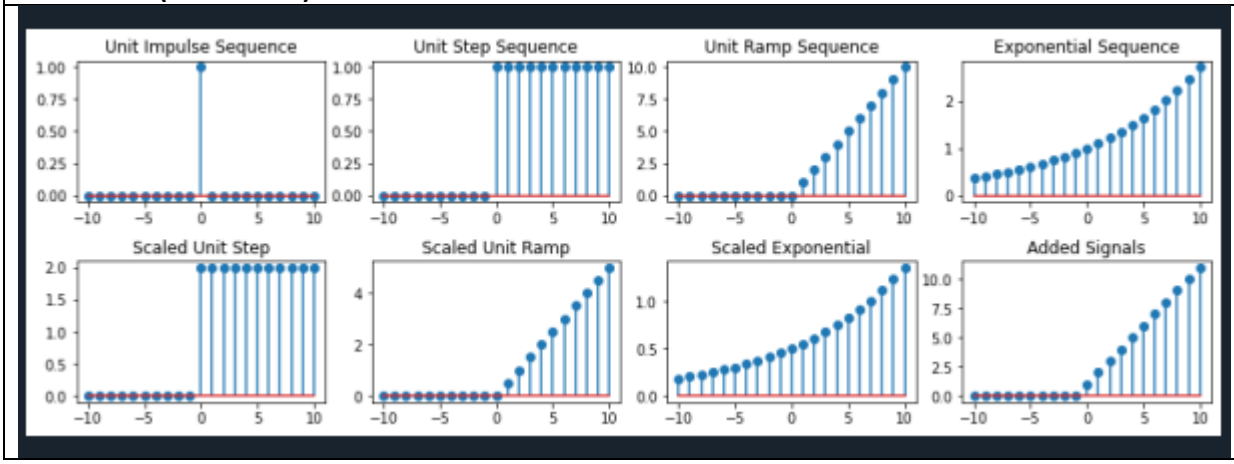
Brief description (3-5 lines)
Run all above lab examples and experiment by changing time interval range, step values. **Basic Signal Operations:** Basic signal operations for discrete-time sequences involve similar concepts to continuous-time signals, but with discrete indices instead of continuous time. **Types of Basic Signal Operations:** 1.Addition/Subtraction 2.Scaling 3.Time Shifting 4.Time Reversal 5.Time Scaling
The code

```
# Basic Signal Operations
import numpy as np
import matplotlib.pyplot as plt
```

```
# Function to generate unit impulse sequence
def unit_impulse(n):
return np.where(n == 0, 1, 0)
# Function to generate unit step sequence
def unit_step(n):
return np.where(n >= 0, 1, 0)
# Function to generate unit ramp sequence
def unit_ramp(n):
return np.where(n >= 0, n, 0)
# Function to generate exponential sequence
def exponential(n, A, a):
return A * np.exp(a * n)
# Sequence generation
n = np.arange(-10, 11) # Define the range of n
A = 1 # Amplitude
a = 0.1 # Exponential growth/decay rate
# Original sequences
impulse = unit_impulse(n)
step = unit_step(n)
ramp = unit_ramp(n)
exponential_seq = exponential(n, A, a)
# Sequence operations
# Scaling
scaled_step = 2 * step
scaled_ramp = 0.5 * ramp
scaled_exponential = 0.5 * exponential_seq
# Time shifting
shifted_step = unit_step(n - 3)
shifted_ramp = unit_ramp(n - 3)
shifted_exponential = exponential(n - 3, A, a)
# Addition
added_signals = step + ramp
# Plotting
plt.figure(figsize=(12, 8))
# Original signals
plt.subplot(4, 4, 1)
plt.stem(n, impulse, use_line_collection=True)
plt.title('Unit Impulse Sequence')
plt.subplot(4, 4, 2)
plt.stem(n, step, use_line_collection=True)
plt.title('Unit Step Sequence')
plt.subplot(4, 4, 3)
plt.stem(n, ramp, use_line_collection=True)
plt.title('Unit Ramp Sequence')
plt.subplot(4, 4, 4)
```

```
plt.stem(n, exponential_seq, use_line_collection=True)
plt.title('Exponential Sequence')
# Modified signals
plt.subplot(4, 4, 5)
plt.stem(n, scaled_step, use_line_collection=True)
plt.title('Scaled Unit Step')
plt.subplot(4, 4, 6)
plt.stem(n, scaled_ramp, use_line_collection=True)
plt.title('Scaled Unit Ramp')
plt.subplot(4, 4, 7)
plt.stem(n, scaled_exponential, use_line_collection=True)
plt.title('Scaled Exponential')
plt.subplot(4, 4, 8)
plt.stem(n, added_signals, use_line_collection=True)
plt.title('Added Signals')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



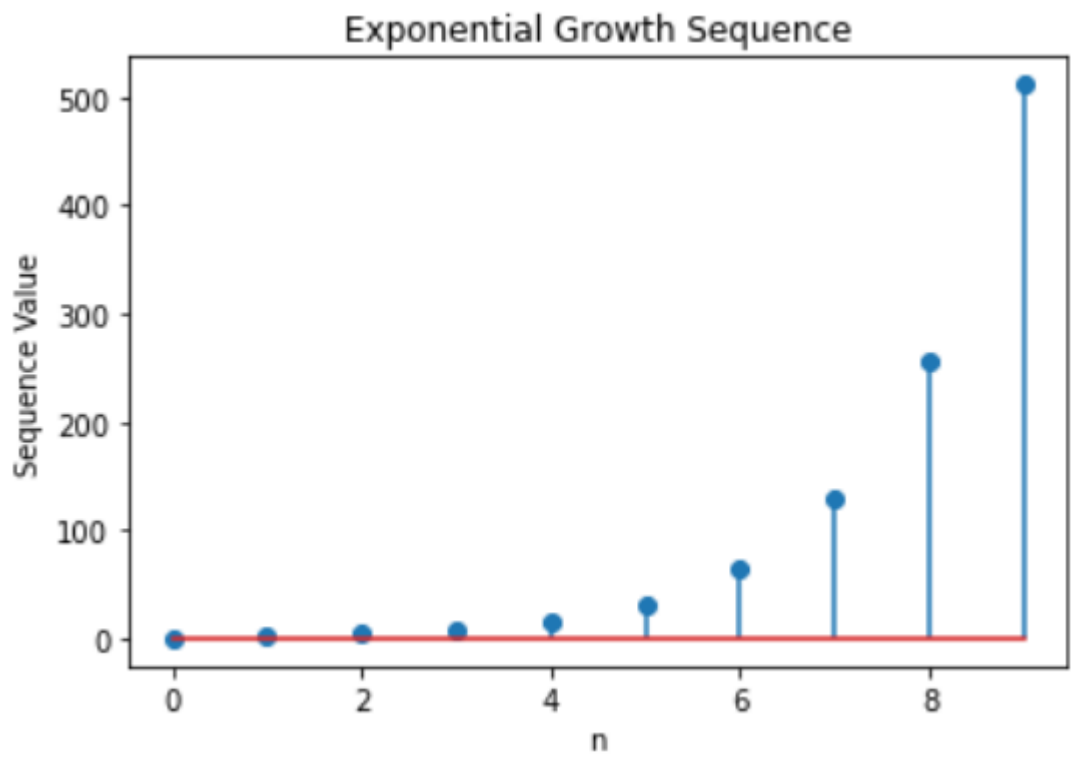
Lab Task No 2(A):

Solution:

Brief description (3-5 lines)
Write python scripts for all possible cases in discrete exponential sequences. Exponential Growth
The code

```python
#Case 1: Exponential Growth
import numpy as np
import matplotlib.pyplot as plt
# Parameters
n = np.arange(0, 10)  # sequence length
a = 2  # base
b = 1  # initial value
# Generate sequence
sequence = b * (a ** n)
# Plot sequence
plt.stem(n, sequence, use_line_collection=True)
plt.title('Exponential Growth Sequence')
plt.xlabel('n')
plt.ylabel('Sequence Value')
plt.show()
```

The results (Screenshot)



Lab Task No 2(B):

Solution:

Brief description (3-5 lines)

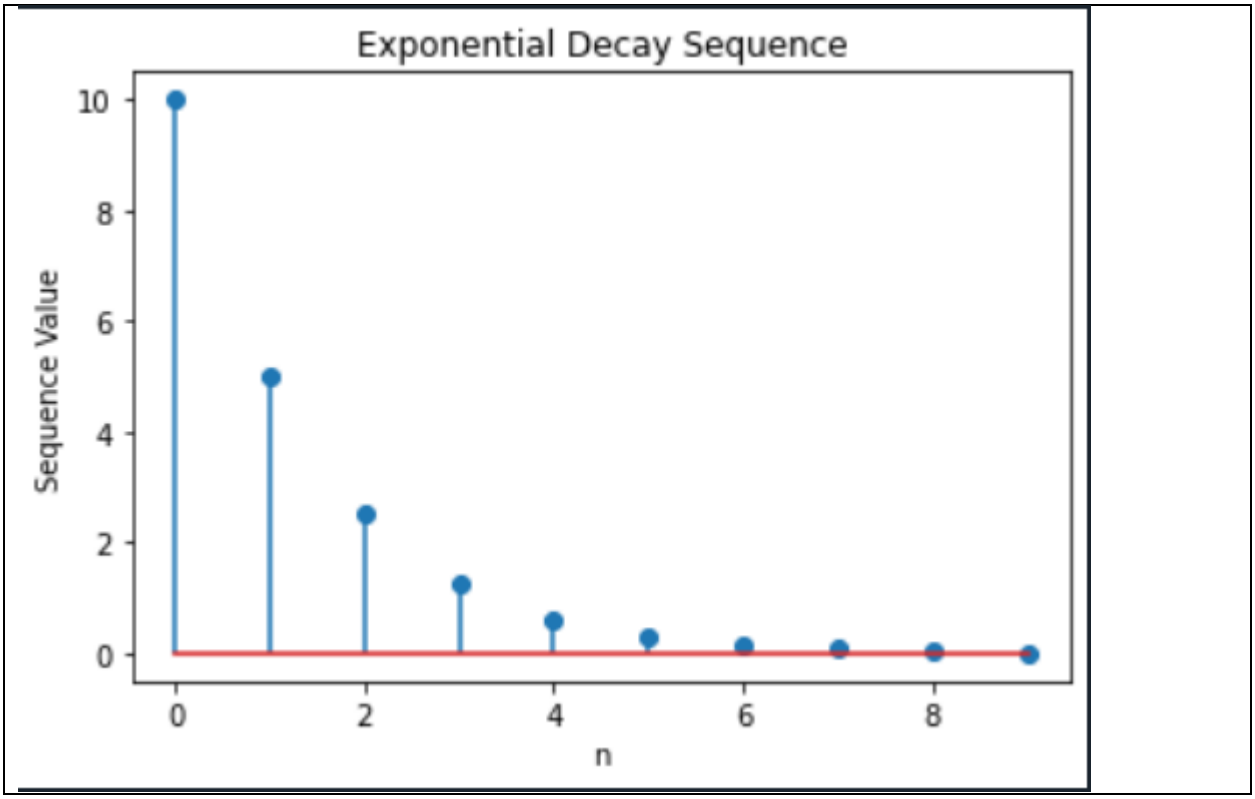
Write python scripts for all possible cases in discrete exponential sequences.

Exponential Decay

The code

```python
# Case 2: Exponential Decay
import numpy as np
import matplotlib.pyplot as plt
# Parameters
n = np.arange(0, 10)  # sequence length
a = 0.5  # base (decay factor)
b = 10  # initial value
# Generate sequence
sequence = b * (a ** n)
# Plot sequence
plt.stem(n, sequence, use_line_collection=True)
plt.title('Exponential Decay Sequence')
plt.xlabel('n')
plt.ylabel('Sequence Value')
plt.show()
```

The results (Screenshot)



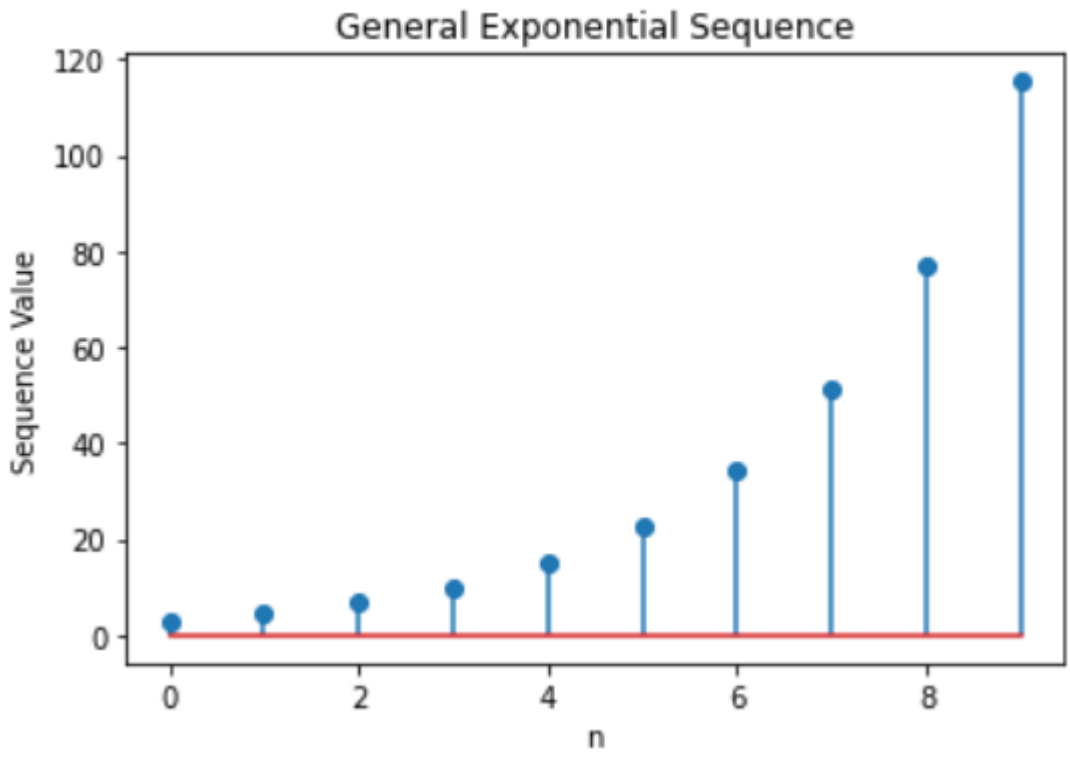
Lab Task No 2(C):

Solution:

Brief description (3-5 lines)
Write python scripts for all possible cases in discrete exponential sequences. General Exponential Sequence
The code

```python
# Case 3: General Exponential Sequence
import numpy as np
import matplotlib.pyplot as plt
# Parameters
n = np.arange(0, 10)  # sequence length
a = 1.5  # base
b = 3  # initial value
# Generate sequence
sequence = b * (a ** n)
# Plot sequence
plt.stem(n, sequence, use_line_collection=True)
plt.title('General Exponential Sequence')
plt.xlabel('n')
plt.ylabel('Sequence Value')
plt.show()
```

The results (Screenshot)



Lab Task No 3:

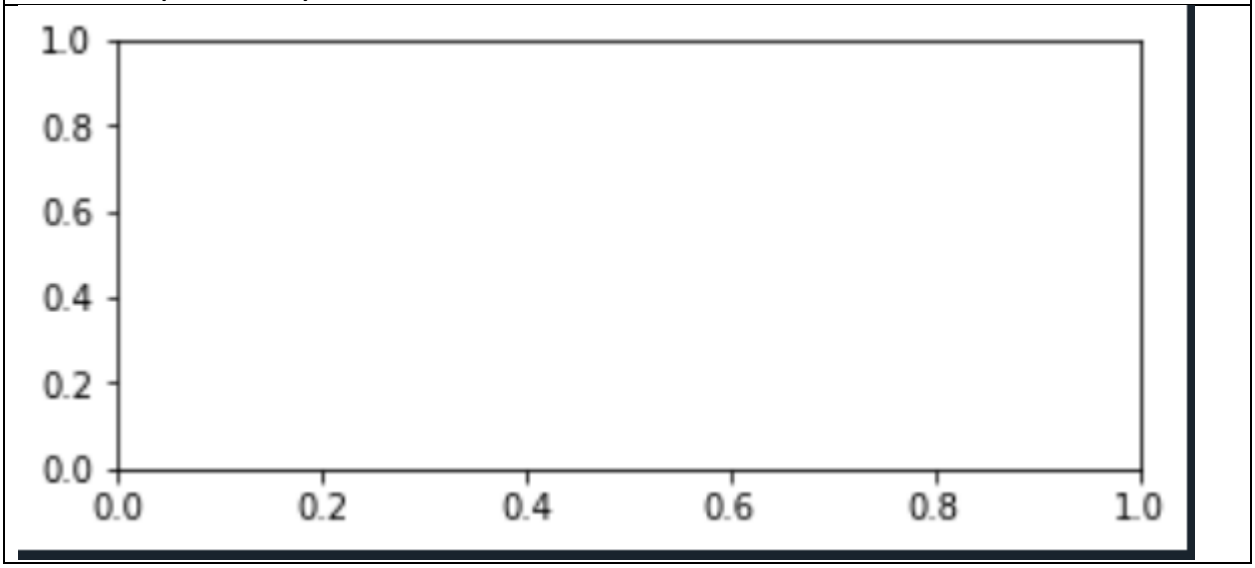
Solution:

Brief description (3-5 lines)
Apply all Signal Operations on above all explained sequences.
The code

```
#SIGNAL OPERATION
import numpy as np
import matplotlib.pyplot as plt
# Define the sequences
n = np.arange(-5, 6)  # Time index
# Define sequences
x1 = np.array([1, 2, 3, 4, 5])  # Sequence 1
x2 = np.array([5, 4, 3, 2, 1])  # Sequence 2
# Apply signal operations
# Addition
addition = x1 + x2
# Subtraction
subtraction = x1 - x2
# Scaling
scaling_factor = 2
scaled_x1 = scaling_factor * x1
# Time shifting
shifted_index = n + 2
time_shifted_x1 = np.roll(x1, 2)
# Reversal
reversed_x1 = np.flip(x1)
# Plotting
plt.figure(figsize=(12, 8))
# Original sequences
plt.subplot(3, 2, 1)
plt.stem(n, x1, use_line_collection=True)
plt.title('Sequence x1')
plt.subplot(3, 2, 2)
plt.stem(n, x2, use_line_collection=True)
plt.title('Sequence x2')
# Addition
plt.subplot(3, 2, 3)
plt.stem(n, addition, use_line_collection=True)
plt.title('Addition x1 + x2')
# Subtraction
plt.subplot(3, 2, 4)
plt.stem(n, subtraction, use_line_collection=True)
plt.title('Subtraction x1 - x2')
# Scaling
plt.subplot(3, 2, 5)
plt.stem(n, scaled_x1, use_line_collection=True)
```

```
plt.title('Scaling 2 * x1')
# Time shifting
plt.subplot(3, 2, 6)
plt.stem(shifted_index, time_shifted_x1, use_line_collection=True)
plt.title('Time Shifted x1')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



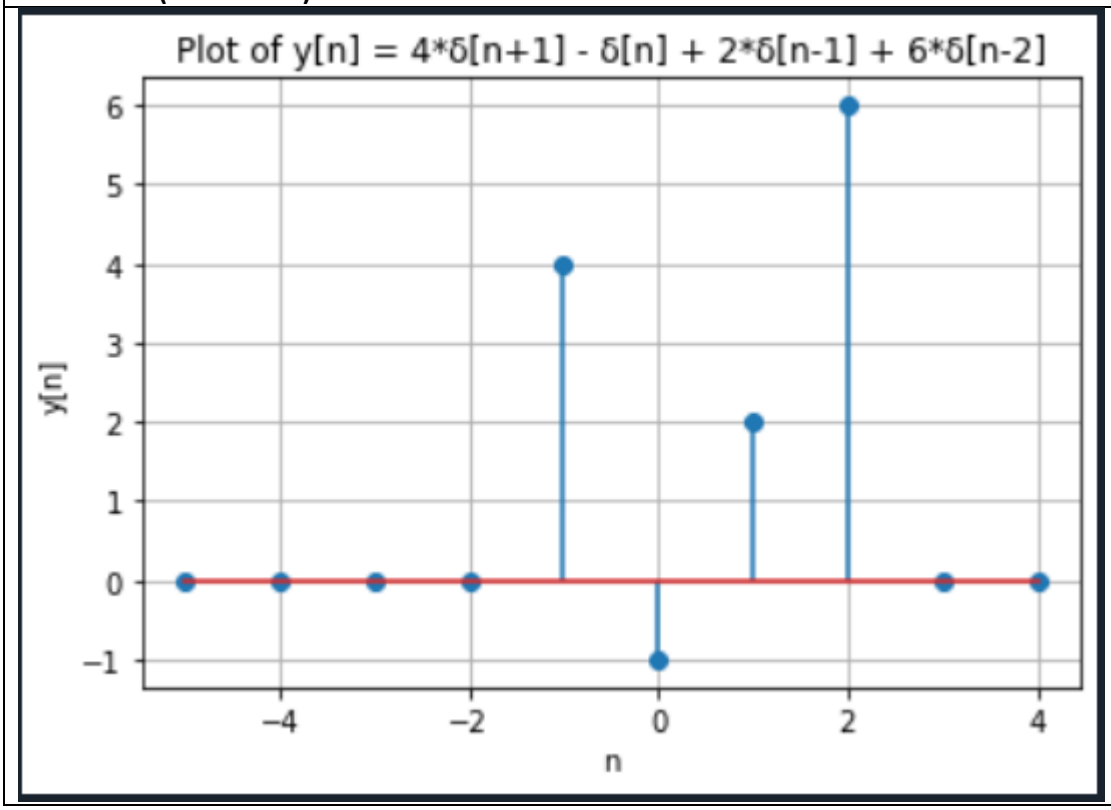
Lab Task No 4:

Solution:

Brief description (3-5 lines)
Plot y[n] = 4*δ[n+1] - δ[n] + 2*δ[n-1] + 6*δ[n-2]
The code

```python
#Plot y[n] = 4*δ[n+1] - δ[n] + 2*δ[n-1] + 6*δ[n-2]
import numpy as np
import matplotlib.pyplot as plt
# Define the unit impulse function
def delta(n):
    return 1 if n == 0 else 0
# Define the unit step function
def u(n):
    return 1 if n >= 0 else 0
# Define the signal function
def y(n):
    return 4 * delta(n + 1) - delta(n) + 2 * delta(n - 1) + 6 * delta(n - 2)
# Define the range of n
n = np.arange(-5, 5)
# Compute the values of the signal y[n]
y_values = [y(i) for i in n]
# Plotting
plt.stem(n, y_values, use_line_collection=True)
plt.xlabel('n')
plt.ylabel('y[n]')
plt.title('Plot of y[n] = 4*δ[n+1] - δ[n] + 2*δ[n-1] + 6*δ[n-2]')
plt.grid(True)
plt.show()
```

The results (Screenshot)



Lab Task No 5:

Solution:

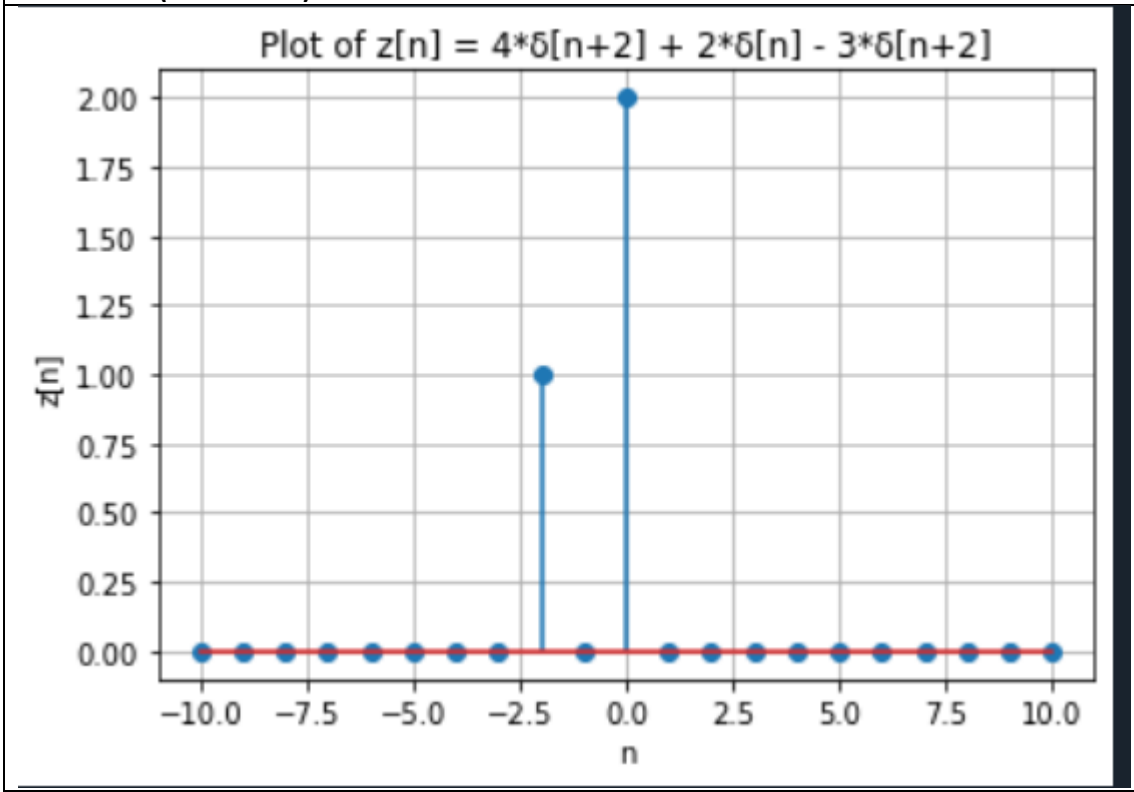
Brief description (3-5 lines)

Plot z[n] = 4*δ[n+2] + 2*δ[n] - 3*δ[n+2]

The code

```python
#Plot z[n] = 4*δ[n+2] + 2*δ[n] - 3*δ[n+2]
import numpy as np
import matplotlib.pyplot as plt
# Define the unit step sequence
def u(n):
    return np.heaviside(n, 1)
# Define the unit impulse signal
def delta(n):
    return np.where(n == 0, 1, 0)
# Define the signal z[n]
def z(n):
    return 4 * delta(n + 2) + 2 * delta(n) - 3 * delta(n + 2)
# Define the range of n
n = np.arange(-10, 11)
# Calculate z[n]
z_n = z(n)
# Plotting
plt.stem(n, z_n, use_line_collection=True)
plt.xlabel('n')
plt.ylabel('z[n]')
plt.title('Plot of z[n] = 4*δ[n+2] + 2*δ[n] - 3*δ[n+2]')
plt.grid(True)
plt.show()
```

The results (Screenshot)



Conclusion
In conclusion, we studied about the Sequences are sorted sets of elements in Python that can be iterated over and indexed. Lists, tuples, and strings are examples of common sequence types; each has unique properties and operations. Sequences facilitate actions such as repetition, concatenation, slicing, and membership testing. Python algorithm creation and effective data manipulation depend on an understanding of sequences. Programmers can improve the readability, maintainability, and performance of their code by utilizing sequences in an efficient manner.